

11-5-26

① Bankers algorithm:

```
#include <stdio.h>
```

```
void input Matrices (int n, int m, int alloc[n][m],  
                    int max[n][m], int avail[m])
```

```
{  
    printf("Enter alloc. matrix (%d x %d): \n", n, m);
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            scanf("%d", &alloc[i][j]);
```

```
    printf("Enter max matrix:");
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            scanf("%d", &max[i][j]);
```

```
    printf("Enter avail. resources (%d values) \n",  
          m);
```

```
    for (int i = 0; i < m; i++)
```

```
        scanf("%d", &avail[i]);
```

```
}
```

```
void calculateNeed (int n, int m, int alloc[n][m],  
                  int max[n][m], int need[n][m])
```

```
int f[n], ans
```

```
{  
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++)
```

```
            need[i][j] = max[i][j]
```

```
                - alloc[i][j];
```

```
void bankersAlgm (int n, int m, int alloc[n][m],  
                 int need[n][m]) {
```

```
    int f[n], ans[n], ind = 0;
```

```
    for (int i = 0; i < n; i++) f[i] = 0;
```

```

for (int k=0; k<n; k++)
    for (int i=0; i<n; i++) {
        if (f[i] == 0) {
            int flag = 0;
            for (int j=0; j<m; j++) {
                if (need[i][j] > avail[j]) {
                    flag = 1;
                    break;
                }
            }
            if (flag == 0) {
                ans[ind++] = i;
                for (int y=0; y<m; y++)
                    avail[y] = alloc[i][y];
                f[i] = 1;
            }
        }
    }
}

```

```

print f(" Safe sequence: ");
for (int i=0; i<n-1; i++)
    print f("P%d -> ", ans[i]);
print f("P%d\n", ans[n-1]);

```

```

}

```

```

int main() {

```

```

    int n, m;
    printf("Enter no. of processes & resources\n");
    scanf("%d %d", &n, &m);

```

```

    int alloc[n][m], max[n][m], avail[m],
        need[n][m];

```

```

    input Matrices (n, m, alloc, max, avail);
    calculate Need (n, m, alloc, max, need);
    bankers Alg (n, m, alloc, need, avail);
    return 0;
}

```


O/P: Enter no. of processes & resources:

5 3

Enter Alloc matrix (5 x 3):

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Max Matrix

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter avail sources:

3 3 2

Safe sequence: $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

12/5/26